

ROYAUME DU MAROC

*_*_*_*_*

HAUT COMMISSARIAT AU PLAN



*_*_*_*_*_*_*_*_*_*

INSTITUT NATIONAL
DE STATISTIQUE ET D'ECONOMIE
APPLIQUEE



INSEA

Projet de IOT

Conception et réalisation d'une serre agricole
connectée et automatisée

Préparé par :

KEBE Mohamed (DSE)

DIALLO Ahmadou Siradiou (M2SI)

DAROUICHE Chaimae (DSE)

KADADE MAHAMADOU Bassirou (M2SI)

Salma Chkoubi (DSE)

Sous la direction de : *Mme CHLIOUI Imane*

Année académique : *2025 – 2026*

Table des matières

Remerciements	5
--------------------------------	----------

Introduction	6
-------------------------------	----------

CADRE THÉORIQUE ET MÉTHODOLOGIQUE

1	Présentation du système et architecture générale	8
1.1	Cadre du projet	8
1.1.1	Cadre académique	8
1.1.2	Contexte applicatif	8
1.2	Présentation détaillée des composants matériels	8
1.2.1	ESP32 — Microcontrôleur principal	9
1.2.2	DHT22 — Température et humidité de l'air	9
1.2.3	HC-SR04 — Niveau d'eau (capteur ultrason)	9
1.2.4	LDR — Luminosité	10
1.2.5	Capteur d'humidité du sol (joystick analogique)	10
1.2.6	NTC — Température du sol	11
1.2.7	MQ135 — Qualité de l'air	11
1.2.8	Afficheur OLED SSD1306 et actionneurs	11
1.3	Bibliothèques logicielles	12
1.4	Architecture globale du système	12
1.5	Environnement de simulation : Wokwi vs Tinkercad	13
1.6	Réalisme de la simulation : pourquoi introduire du bruit dans les mesures ?	14
1.7	Répartition des tâches et méthodologie	15
2	Acquisition des données et logique de contrôle automatique	16
2.1	Schéma de câblage	16
2.2	Lecture des capteurs	17
2.2.1	DHT22 — Température et humidité de l'air	17
2.2.2	Capteurs analogiques — LDR, sol et qualité de l'air	17
2.2.3	HC-SR04 — Niveau d'eau	18
2.3	Logique de décision automatique	18
2.4	Affichage local sur écran OLED	19
2.5	Validation sur l'environnement Wokwi	19

CADRE PRATIQUE

3	Communication MQTT et transmission des données	22
3.1	Introduction	22
3.2	Concepts fondamentaux	22
3.2.1	Le protocole MQTT	22
3.3	Implémentation	22
3.3.1	Paramètres de connexion WiFi et MQTT	22
3.3.2	Organisation des topics	23
3.3.3	Connexion WiFi	23
3.3.4	Connexion MQTT	23
3.3.5	Publication des données	24
3.3.6	Rythme de publication	25
3.4	Vérification et tests	25
3.5	Librairies et outils utilisés	26
4	Dashboard Node-RED et stockage des données	27
4.1	Mise en place de l'environnement Node-RED	27
4.2	Réception des données MQTT	28
4.3	Vérification et débogage des données reçues	29
4.4	Construction du dashboard Node-RED	30
4.4.1	Affichage des conditions climatiques	30
4.4.2	Affichage des états des actionneurs	32
4.5	Graphique historique multi-courbes	33
4.6	Stockage historique avec SQLite	33
4.6.1	Création de la table	33
4.6.2	Insertion automatique des mesures	34
4.6.3	Vérification des données stockées	35
4.7	Organisation générale du flux Node-RED	35
5	Agent LLM et analyse intelligente des conditions de culture	38
5.1	Vue d'ensemble	38
5.2	Partie A — Backend Flask	38
5.2.1	Rôle du backend	38
5.2.2	Routes REST exposées	38
5.2.3	Algorithme de calcul du score de santé	39
5.3	Partie B — Architecture Globale — Pipeline à Quatre Couches	39
5.4	Partie C — Intégration de l'Agent LLM SmartAssist	39
5.4.1	Pipeline d'intégration IA — Étape par Étape	39
5.5	Interface du Dashboard — Vue en Temps Réel	40
5.6	Interface SmartAssist — Agent Conversationnel	41
5.7	Scénario d'exécution réel — Diagnostic de luminosité	41

5.8	Avantages et Plus-value	41
	Conclusion générale	42
	Références et supports utilisés	43

Remerciements

Au terme de ce projet, nous tenons à exprimer notre profonde gratitude à l'ensemble des personnes qui ont contribué, de près ou de loin, à sa réalisation.

Nous adressons en premier lieu nos remerciements les plus sincères à notre **enseignant encadrant**, pour la pertinence de ses orientations, sa disponibilité et la rigueur méthodologique qu'elle nous a transmise tout au long de ce travail. Son accompagnement a constitué un véritable cadre professionnel, comparable à celui d'une conduite de projet en entreprise.

Nos remerciements s'adressent également à l'ensemble du **corps enseignant** du Master *Systèmes Intelligents* (M2SI) et de la 2^{ème} année *Data Software Engineering* (DSE), dont la qualité de la formation dispensée nous a permis d'acquérir les compétences techniques et organisationnelles mobilisées dans ce projet.

Nous saluons enfin l'esprit d'équipe et l'engagement de chacun des **membres du groupe**, dont la complémentarité et la coordination ont rendu possible la conduite de ce projet dans le respect des objectifs et des délais fixés. Nous dédions ce travail à nos familles et à nos proches, pour leur soutien constant.

Introduction

L'agriculture de précision figure aujourd'hui parmi les secteurs où l'Internet des Objets apporte la plus forte valeur ajoutée. Face aux enjeux de sécurité alimentaire, de rarefaction de l'eau et de maîtrise des coûts d'exploitation, les producteurs recherchent des solutions capables de surveiller sans interruption le micro-climat de leurs cultures, de déclencher d'elles-mêmes l'arrosage ou l'éclairage, et de transformer les mesures relevées en informations directement exploitables. La serre, espace clos où chaque paramètre peut être maîtrisé, offre le terrain d'application idéal de ces technologies.

Toute la difficulté consiste alors à bâtir, à coût raisonnable et de façon reproductible, un dispositif embarqué qui perçoit en continu les grandeurs déterminantes d'une culture, qui réagit seul lorsque ces grandeurs s'écartent des conditions favorables, et qui restitue son état d'une manière compréhensible aussi bien pour l'exploitant que pour un outil d'analyse. C'est à cette exigence que répond le présent travail.

Conçu autour d'un microcontrôleur ESP32, le prototype décrit ici recrée une serre vouée à la culture de tomates. Une chaîne de six capteurs y relève la température et l'humidité de l'air, la luminosité, l'humidité et la température du sol, le niveau d'eau du réservoir et la qualité de l'air ; en réponse, une pompe d'arrosage, plusieurs indicateurs lumineux et un afficheur OLED traduisent localement les décisions du système. Cet ensemble matériel constitue le socle d'une plateforme plus vaste, qui prolonge la mesure par la communication, la supervision à distance et l'aide à la décision. Le projet est le fruit d'une collaboration entre les étudiants du **Master Systèmes Intelligents (M2SI)** et de la **2^{ème} année Data Software Engineering (DSE)**, réunis autour du module *Projet IoT*.

Le lecteur découvrira d'abord le cadre du projet, l'inventaire détaillé du matériel et le choix de l'environnement de simulation, puis la manière dont les données sont acquises, câblées et exploitées pour piloter automatiquement la serre. Ces deux étapes fondatrices ouvrent ensuite la voie à la communication des données, à leur visualisation, à leur archivage et à leur interprétation intelligente, traités dans la suite du rapport.

CADRE THÉORIQUE ET MÉTHODOLOGIQUE

Ce premier chapitre pose les fondations du projet. Il précise le cadre dans lequel il s'inscrit, présente en détail chacun des composants matériels retenus, recense les bibliothèques logicielles mobilisées, décrit l'architecture globale de la solution et justifie le choix de l'environnement de simulation à l'issue d'un comparatif entre les deux plateformes les plus répandues.

1.1 Cadre du projet

1.1.1 Cadre académique

Ce projet s'inscrit dans le module *Projet IoT* et résulte d'une collaboration entre les étudiants du Master *Systèmes Intelligents* (M2SI) et de la 2^{ème} année *Data Software Engineering* (DSE). Il vise à mettre en pratique, dans une démarche proche d'un cahier des charges industriel, l'ensemble de la chaîne IoT : **acquisition** de données par capteurs, **traitement** embarqué sur microcontrôleur, **automatisation** d'actionneurs et **restitution** de l'information. Au-delà de la maîtrise technique de l'ESP32 et de son écosystème, les objectifs pédagogiques portent sur la conduite de projet en équipe, la répartition des responsabilités et la production d'une documentation de qualité professionnelle.

1.1.2 Contexte applicatif

Le système modélise une **serre destinée à la culture de tomates**, plante exigeante dont le rendement dépend étroitement de la température, de l'humidité de l'air et du sol, de la luminosité et de la qualité de l'air. Le prototype reproduit ces conditions à l'aide de six capteurs et réagit automatiquement aux écarts détectés : déclenchement de la **pompe d'arrosage** lorsque le niveau d'eau du réservoir est insuffisant, et signalisation par **indicateurs lumineux** des situations d'air trop sec, de tombée de la nuit ou de concentration excessive en CO₂. Le projet est conduit par une équipe de cinq membres dont les responsabilités sont détaillées en fin de chapitre.

1.2 Présentation détaillée des composants matériels

Le cœur du système est le microcontrôleur **ESP32**, retenu pour sa puissance de calcul, sa connectivité Wi-Fi/Bluetooth intégrée, son grand nombre de broches et ses convertisseurs analogique-numérique (ADC) sur 12 bits (valeurs de 0 à 4095). Autour de lui gravitent six capteurs, un afficheur et quatre actionneurs, présentés ci-après.

1.2.1 ESP32 — Microcontrôleur principal

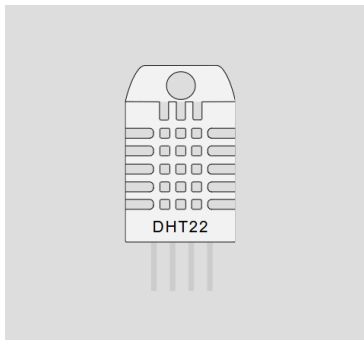


Type	Microcontrôleur 32 bits double cœur (Xtensa LX6)
Connectivité	Wi-Fi 802.11 b/g/n et Bluetooth intégrés
ADC	12 bits (0 – 4095) sur les broches analogiques
Rôle	Acquisition, traitement, contrôle des actionneurs et affichage

IMAGE 1 – Carte de développement ESP32 DevKit v4.

1.2.2 DHT22 — Température et humidité de l'air

Capteur numérique communiquant sur un seul fil (protocole propriétaire mono-fil). Il mesure simultanément la température et l'humidité relative de l'air ambiant de la serre. Il est relié à la broche GPI04.



Grandeurs	Température et humidité de l'air
Plages	−40 à +80 °C ($\pm 0,5$ °C) ; 0 à 100 % HR (± 2 à 5 %)
Interface	Numérique mono-fil (1-Wire propriétaire)
Broche	GPI04 (DATA), alimentation 3,3 V

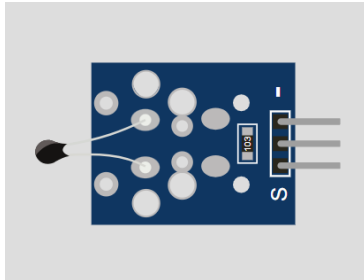
IMAGE 2 – Capteur DHT22 (température et humidité de l'air).

1.2.3 HC-SR04 — Niveau d'eau (capteur ultrason)

Capteur de distance à ultrasons (40 kHz) utilisé ici pour estimer le **niveau d'eau du réservoir** : il mesure la distance entre le capteur et la surface de l'eau. Une distance élevée traduit un réservoir presque vide. Il utilise les broches GPI018 (TRIG) et GPI05 (ECHO).

1.2.6 NTC — Température du sol

Thermistance à coefficient de température négatif (NTC) : sa résistance varie avec la température. Elle est lue sur GPI013 et convertie en température du sol (plage 10 à 50 °C).



Grandeur	Température du sol
Sortie	Analogique → 10 à 50 °C
Principe	Résistance variable avec la température (NTC)
Broche	GPI013 (sortie OUT)

IMAGE 6 – Thermistance NTC (température du sol).

1.2.7 MQ135 — Qualité de l'air

Capteur de gaz sensible au CO₂, à l'ammoniac, au benzène et à la fumée. Sa sortie analogique, lue sur GPI035, est convertie en concentration équivalente (200 à 2000 ppm) afin de surveiller la qualité de l'air de la serre.



Grandeur	Qualité de l'air (CO ₂ , NH ₃ , fumée...)
Sortie	Analogique → 200 à 2000 ppm
Rôle	Détection d'une concentration excessive en CO ₂
Broche	GPI035 (sortie analogique AO)

IMAGE 7 – Capteur de gaz MQ135 (qualité de l'air).

1.2.8 Afficheur OLED SSD1306 et actionneurs

L'écran **OLED SSD1306** (128×64 pixels, bus I²C, adresse 0x3C) assure l'affichage local des mesures sur deux pages alternées. Le système comporte par ailleurs quatre actionneurs récapitulés dans le tableau 1.

Actionneur	Broche	Rôle / condition d'activation
Relais pompe	GPI02	Arrosage — actif à l'état BAS ; activé si le niveau d'eau est bas
LED verte	GPI025	Alerte air sec (humidité de l'air sous le seuil)
LED bleue	GPI014	Indicateur de nuit (luminosité faible)
LED rouge	GPI027	Alerte CO₂ élevé (qualité d'air dégradée)
Écran OLED	GPI021/22	Affichage local (I ² C : SDA, SCL)

TABLE 1 – Afficheur et actionneurs du système.

1.3 Bibliothèques logicielles

Le développement repose sur le framework Arduino pour ESP32 et sur un ensemble de bibliothèques open source. Le tableau 2 en précise le rôle.

Bibliothèque	Rôle
DHT sensor library	Lecture du capteur DHT22 (température et humidité)
Adafruit GFX Library	Primitives graphiques pour l'affichage
Adafruit SSD1306	Pilote de l'écran OLED I ² C
OneWire	Bus mono-fil (sondes de température de type DS18B20)
DallasTemperature	Exploitation des sondes DS18B20
PubSubClient	Client MQTT (utilisé au chapitre 3)
WiFi	Connectivité réseau de l'ESP32

TABLE 2 – Bibliothèques logicielles installées dans le projet.

1.4 Architecture globale du système

L'architecture suit le schéma classique d'une chaîne IoT en quatre étages. Le tableau 3 synthétise ces composants.

Étage	Composant	Rôle
Perception	DHT22, HC-SR04, LDR, NTC, joystick, MQ135	Acquisition des grandeurs physiques
Traitement	ESP32	Logique de décision, pilotage des actionneurs
Restitution locale	OLED, LEDs, pompe	Affichage et actions sur la serre
Communication	Wi-Fi, MQTT	Transmission des données
Supervision & décision	Node-RED, SQLite, LLM	Dashboard, historique, analyse

TABLE 3 – Architecture en couches du système de serre connectée.

1.5 Environnement de simulation : Wokwi vs Tinkercad

Critère	Wokwi	Tinkercad Circuits
Support ESP32	Oui (natif, avec Wi-Fi)	Non (Arduino Uno, Mega, micro :bit)
Connectivité Wi-Fi / MQTT	Simulée (passerelle Internet)	Non disponible
Capteurs disponibles	Très large (DHT22, HC-SR04, MQ-x, OLED I ² C...)	Catalogue plus restreint
Écran OLED SSD1306	Pris en charge nativement	Non pris en charge
Éditeur de code	IDE complet, bibliothèques Arduino	Éditeur bloc / texte simplifié
Moniteur série	Oui, fidèle	Oui, basique
Public cible	Prototypage IoT avancé	Initiation à l'électronique
Coût	Gratuit (offre Pro optionnelle)	Gratuit

TABLE 4 – Comparatif des environnements de simulation Wokwi et Tinkercad.

Conclusion du comparatif. Tinkercad Circuits ne prend pas en charge l'ESP32, ni la connectivité Wi-Fi/MQTT, ni l'écran OLED SSD1306 — trois éléments indispensables à

notre architecture. **Wokwi** a donc été retenu car il simule fidèlement l'ESP32 et l'intégralité de notre chaîne de capteurs et d'actionneurs, tout en autorisant une communication réseau réaliste.

1.6 Réalisme de la simulation : pourquoi introduire du bruit dans les mesures ?

Dans un simulateur, un capteur configuré sur une valeur donnée renvoie exactement cette même valeur à chaque lecture. Or aucun capteur réel ne se comporte ainsi. Pour rendre la simulation crédible, le firmware reconstitue ces phénomènes au moyen de **trois mécanismes complémentaires** :

- un **micro-bruit** aléatoire de faible amplitude, ajouté à chaque cycle de lecture ;
- des **offsets** réévalués lentement, toutes les 15 à 25 secondes, qui reproduisent la dérive progressive des conditions de la serre ;
- une **contrainte de cohérence physique** : les offsets sont bornés en fonction du scénario météo, de sorte que les mesures restent plausibles et cohérentes entre elles.

```
1 // Mesure finale = valeur brute + derive lente (offset) + micro-bruit
2 tempAir = tR + offTempAir + bruitTemp;
3 humAir  = constrain(hR + offHumAir + bruitHum, 0.0f, 100.0f);
4 // Les offsets sont bornes selon le scenario meteo (coherence physique)
```

Listing 1 – Simulation réaliste : bruit + offset + cohérence physique

1.7 Répartition des tâches et méthodologie

Membre	Responsabilité principale	Livrables
1	Simulation Wokwi et capteurs	Projet Wokwi : ESP32 et chaîne de capteurs, lectures, Serial Monitor
2	Logique automatique, OLED et LEDs	Pilotage pompe et LEDs, affichage OLED, conditions de décision
3	Communication MQTT	Connexion Wi-Fi, broker MQTT, publication des topics
4	Dashboard Node-RED et SQLite	Réception MQTT, jauges, graphiques, stockage historique
5	Agent LLM, intégration et rapport	Seuils idéaux tomates, logique LLM, rapport final, démonstration

TABLE 5 – Répartition des tâches entre les membres de l'équipe.

Conclusion du chapitre

Ce premier chapitre a posé les **fondations** du projet : cadre académique et applicatif, chaîne complète de **six capteurs**, **bibliothèques logicielles**, **architecture en couches** et choix justifié de **Wokwi**. Le chapitre suivant passe à la mise en œuvre concrète avec le schéma de câblage, la lecture effective des capteurs et la logique de contrôle automatique.

Ce chapitre décrit la mise en œuvre concrète du système : le **schéma de câblage**, la **lecture** de chaque capteur, la **logique de décision automatique** qui pilote la pompe et les indicateurs lumineux, ainsi que l'**affichage OLED** et la **validation** sur Wokwi.

2.1 Schéma de câblage

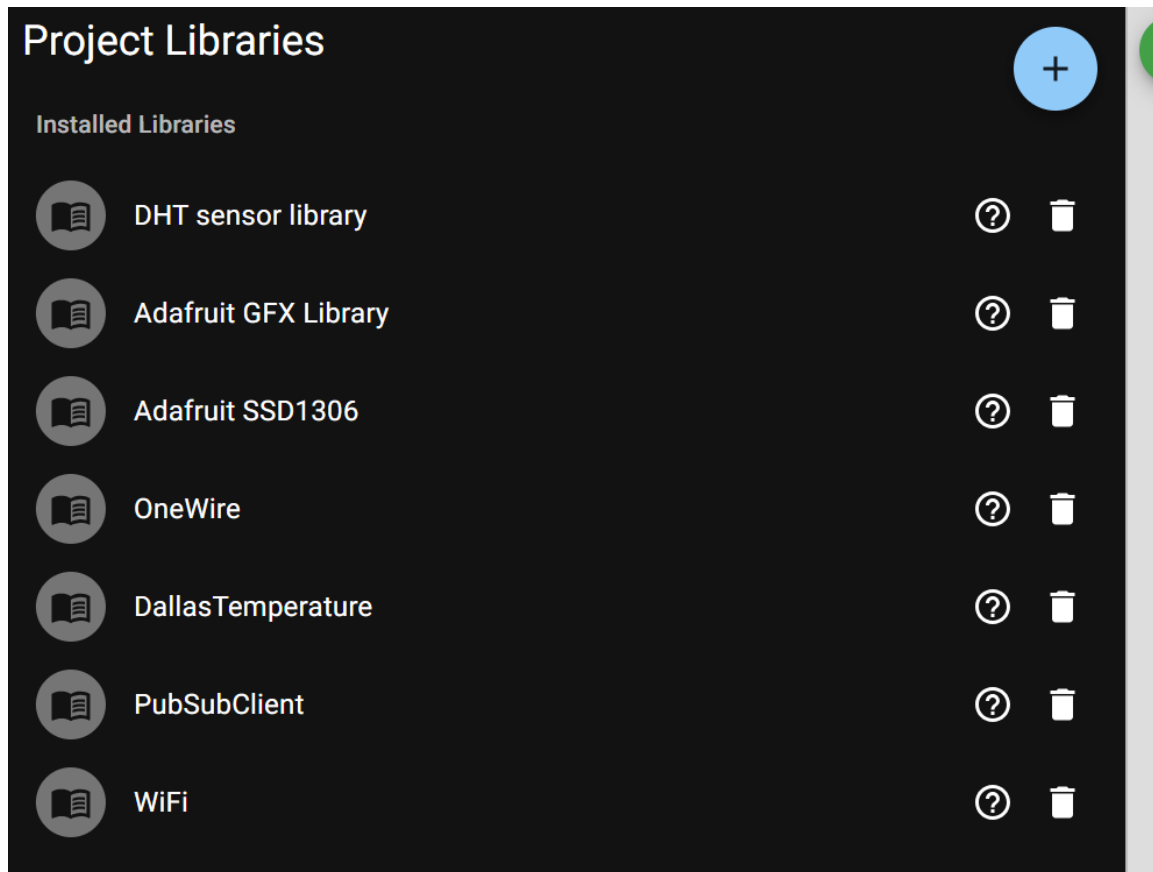


IMAGE 8 – Schéma de câblage du système sous Wokwi (ESP32, capteurs et actionneurs).

Composant	Broche ESP32	Signal / remarque
DHT22	GPI04	Donnée numérique (température/humidité air)
HC-SR04 (TRIG)	GPI018	Déclenchement de l'impulsion ultrason
HC-SR04 (ECHO)	GPI05	Réception de l'écho
LDR	GPI026	Entrée analogique (luminosité)
Joystick (sol)	GPI034	Entrée analogique (humidité sol)
NTC	GPI013	Entrée analogique (température sol)
MQ135	GPI035	Entrée analogique (qualité air)
OLED SDA	GPI021	Bus I ² C (données)
OLED SCL	GPI022	Bus I ² C (horloge)
Relais pompe	GPI02	Sortie numérique, active à l'état BAS
LED verte	GPI025	Sortie numérique (alerte air sec)
LED bleue	GPI014	Sortie numérique (indicateur nuit)
LED rouge	GPI027	Sortie numérique (alerte CO ₂)

TABLE 6 – Brochage complet des capteurs et actionneurs sur l'ESP32.

2.2 Lecture des capteurs

2.2.1 DHT22 — Température et humidité de l'air

```

1 float tR = dht.readTemperature();
2 float hR = dht.readHumidity();
3 if (isnan(tR)) tR = 25.0f;    // valeur de repli si lecture invalide
4 if (isnan(hR)) hR = 60.0f;
```

Listing 2 – Lecture DHT22 avec valeur de repli

2.2.2 Capteurs analogiques — LDR, sol et qualité de l'air

```

1 int    ldrR    = analogRead(LDR_PIN);                                // 0
   - 4095
2 float humSolR = (analogRead(SOIL_PIN) / 4095.0f) * 100.0f;          // 0 -
   100 %
3 float tSolR   = 10.0f + (analogRead(NTC_PIN) / 4095.0f) * 40.0f;    // 10 -
   50 C
4 float ppmR    = 200.0f + (analogRead(MQ135_PIN)/4095.0f)*1800.0f;    // 200
   - 2000 ppm
```

Listing 3 – Lecture et conversion des capteurs analogiques

2.2.3 HC-SR04 — Niveau d'eau

```

1 digitalWrite(TRIG_PIN, HIGH);
2 delayMicroseconds(10);
3 digitalWrite(TRIG_PIN, LOW);
4 long  duree = pulseIn(ECHO_PIN, HIGH, 30000UL);
5 float distR = (duree > 0) ? (duree * 0.0343f) / 2.0f : 15.0f;    // cm

```

Listing 4 – Mesure de distance par ultrason

2.3 Logique de décision automatique

Grandeur surveillée	Seuil	Action déclenchée
Niveau d'eau (distance)	distance > 20 cm	Pompe ON (réservoir bas)
Humidité de l'air	humidité < 40 %	LED verte ON (alerte air sec)
Luminosité (LDR)	LDR < 500	LED bleue ON (nuit détectée)
Qualité de l'air	CO ₂ > 800 ppm	LED rouge ON (alerte CO ₂ élevé)

TABLE 7 – Seuils et actions de la logique de contrôle automatique.

```

1 // Pompe : ON si le niveau d'eau est bas (relais actif LOW)
2 pompe = (distEau > SEUIL_EAU);
3 digitalWrite(POMPE_PIN, pompe ? LOW : HIGH);
4
5 // Indicateurs lumineux
6 digitalWrite(LED_VERT, (humAir < SEUIL_HUM_AIR) ? HIGH : LOW);
7 digitalWrite(LED_BLEU, (ldr < SEUIL_NUIT) ? HIGH : LOW);
8 digitalWrite(LED_ROUGE, (qualiteAir > SEUIL_CO2ALERTE) ? HIGH : LOW);

```

Listing 5 – Logique de contrôle : pompe et indicateurs lumineux

```

1 if (ldr < SEUIL_NUIT) return "NUIT";
2 else if (ldr < SEUIL_NUAGEUX) return "NUAGEUX";
3 else return "ENSOLEILLE";

```

Listing 6 – Détermination du scénario météo

2.4 Affichage local sur écran OLED

Page	Informations affichées
Page 1	Température et humidité de l'air, état du niveau d'eau (POMPE ON / NIV. OK), scénario météo
Page 2	Température et humidité du sol, taux de CO ₂ (ppm), état de l'air (BON / DÉGRADÉ)

TABLE 8 – Contenu des deux pages de l'affichage OLED.

2.5 Validation sur l'environnement Wokwi

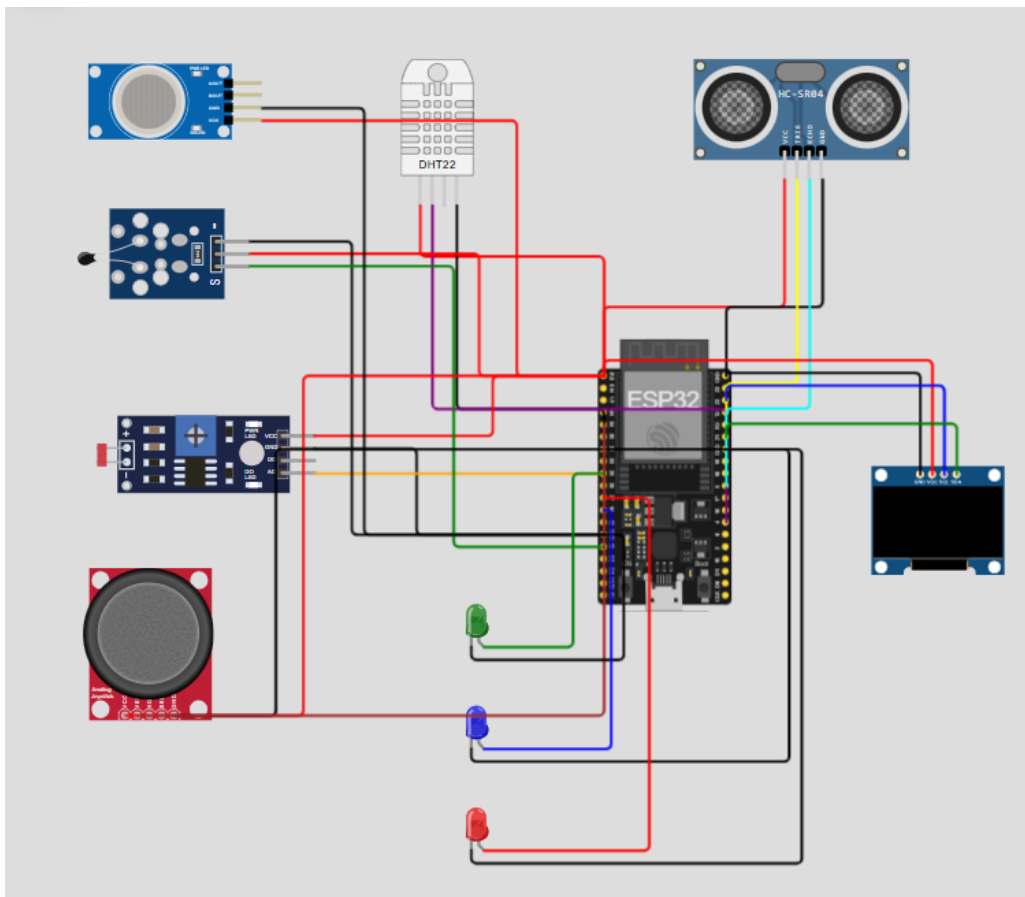


IMAGE 9 – Sortie du moniteur série lors de la simulation (mesures et état des actionneurs).

Conclusion du chapitre

Ce chapitre a concrétisé la **chaîne d'acquisition et de contrôle** : le schéma de câblage et le brochage complet ont été établis, la lecture de chaque capteur a été détaillée, et la logique de décision automatique pilotant la **pompe** et les **trois indicateurs lumineux**

a été formalisée et validée sur Wokwi. La suite du projet portera sur la **transmission de ces données vers l'extérieur** au moyen du protocole **MQTT** (chapitre 3).

CADRE PRATIQUE

3.1 Introduction

Dans le cadre du projet de serre agricole intelligente dédiée à la culture de tomates, la communication entre l'ESP32 et le tableau de bord Node-RED est assurée par le protocole **MQTT** (*Message Queuing Telemetry Transport*).

Cette partie couvre trois responsabilités principales :

- la connexion de l'ESP32 au réseau WiFi Wokwi ;
- le choix et la configuration d'un broker MQTT public ;
- la publication des données sur des topics organisés.

3.2 Concepts fondamentaux

3.2.1 Le protocole MQTT

MQTT est un protocole de messagerie **léger** et **asynchrone**, basé sur un modèle *Publish/Subscribe*, particulièrement adapté aux systèmes embarqués à ressources limitées comme l'ESP32.



Acteur	Rôle
Publisher	L'ESP32 publie les mesures des capteurs sur des topics.
Broker	broker.hivemq.com reçoit et redistribue les messages.
Subscriber	Node-RED s'abonne aux topics et affiche les données.

3.3 Implémentation

3.3.1 Paramètres de connexion WiFi et MQTT

```
1 #define WIFI_SSID      "Wokwi-GUEST"
2 #define WIFI_PASSWORD ""
3 #define MQTT_BROKER    "broker.hivemq.com"
4 #define MQTT_PORT      1883
5 #define MQTT_CLIENT     "serre_esp32_001"
```

Listing 7 – Paramètres WiFi et MQTT

3.3.2 Organisation des topics

```
1 #define TOPIC_TEMPERATURE "serre/temperature"
2 #define TOPIC_HUMIDITE "serre/humidite"
3 #define TOPIC_LUMINOSITE "serre/luminosite"
4 #define TOPIC_ARROSAGE "serre/arrosage"
5 #define TOPIC_ECLAIRAGE "serre/eclairage"
6 #define TOPIC_POMPE "serre/pompe/etat"
7 #define TOPIC_METEO "serre/meteo/scenario"
8 #define TOPIC_JSON "serre/data/json"
```

Listing 8 – Définition des topics MQTT

Topic	Type	Exemple de valeur
serre/temperature	float	27.3
serre/humidite	float	61.2
serre/luminosite	entier	1823
serre/arrosage	état	ON / OFF
serre/eclairage	état	ON / OFF
serre/pompe/etat	état	ON / OFF
serre/meteo/scenario	texte	NUIT / NUAGEUX / ENSOLEILLE
serre/data/json	JSON	{"temp_air":27.3,...}

3.3.3 Connexion WiFi

```
1 void connecterWiFi() {
2     Serial.print("Connexion WiFi");
3     WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
4     int t = 0;
5     while (WiFi.status() != WL_CONNECTED && t < 20) {
6         delay(500); Serial.print("."); t++;
7     }
8     Serial.println(WiFi.status() == WL_CONNECTED
9         ? "\nWiFi connecte - IP: " + WiFi.localIP().toString()
10        : "\nWiFi non disponible - mode hors-ligne");
11 }
```

Listing 9 – Fonction de connexion WiFi

3.3.4 Connexion MQTT

```
1 void connecterMQTT() {
2     if (WiFi.status() != WL_CONNECTED) return;
3     int t = 0;
4     while (!mqtt.connected() && t < 5) {
5         Serial.print("MQTT... ");
6         if (mqtt.connect(MQTT_CLIENT)) {
7             Serial.println("OK");
8         } else {
9             Serial.printf("Echec (code=%d)\n", mqtt.state());
10            delay(1000); t++;
11        }
12    }
13 }
```

Listing 10 – Fonction de connexion MQTT avec retry

3.3.5 Publication des données

```
1 void publierMQTT() {
2     if (!mqtt.connected()) connecterMQTT();
3     if (!mqtt.connected()) return;
4
5     mqtt.publish(TOPIC_POMPE,      pompe ? "ON" : "OFF");
6     mqtt.publish(TOPIC_METEO,      scenarioMeteo().c_str());
7     mqtt.publish(TOPIC_ARROSAGE,    pompe ? "ON" : "OFF");
8     mqtt.publish(TOPIC_ECLAIRAGE,   (ldr < SEUIL_NUIT) ? "ON" : "OFF");
9
10    char buf[16];
11    dtostrf(tempAir, 4, 1, buf);
12    mqtt.publish(TOPIC_TEMPERATURE, buf);
13
14    dtostrf(humAir, 4, 1, buf);
15    mqtt.publish(TOPIC_HUMIDITE, buf);
16
17    itoa(ldr, buf, 10);
18    mqtt.publish(TOPIC_LUMINOSITE, buf);
19
20    char json[400];
21    snprintf(json, sizeof(json),
22             "{\"temp_air\" : %.1f, \"hum_air\" : %.1f, \"meteo\" : \"%s\"}",
23             tempAir, humAir, scenarioMeteo().c_str());
24    mqtt.publish(TOPIC_JSON, json);
25 }
```

Listing 11 – Fonction de publication MQTT — 8 topics

3.3.6 Rythme de publication

```
1 if (millis() - tMQTT >= INTERVALLE_MQTT) {  
2     publierMQTT();  
3     tMQTT = millis();  
4 }
```

Listing 12 – Déclenchement non-bloquant toutes les 5 secondes

QoS utilisé : Le niveau de Qualité de Service choisi est **QoS 0** (*at most once*). Ce niveau est suffisant pour des données environnementales publiées fréquemment : la perte occasionnelle d'un relevé n'est pas critique car le prochain sera publié 5 secondes plus tard.

3.4 Vérification et tests

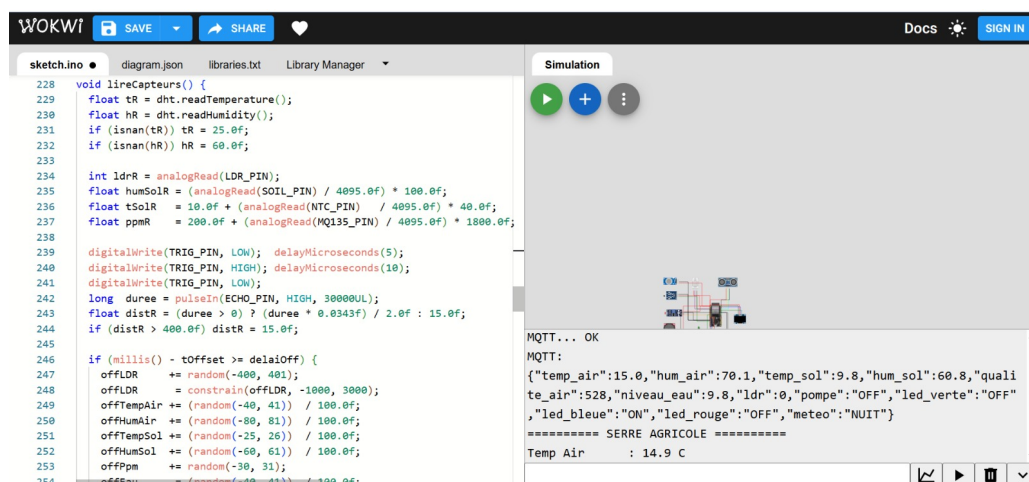


IMAGE 10 – Capture du Serial Monitor — connexion MQTT réussie et publication des données

3.5 Bibliothèques et outils utilisés

Outil / Bibliothèque	Langage	Rôle
PubSubClient	C++ (Arduino)	Client MQTT côté ESP32
WiFi.h	C++ (Arduino)	Connexion réseau Wi-Fi de l'ESP32
HiveMQ paho-mqtt	Broker public Python	Routage des messages MQTT Abonnement MQTT côté backend Flask

Conclusion du chapitre

La partie communication MQTT est pleinement opérationnelle. L'ESP32 publie toutes les 5 secondes les données des capteurs sur **8 topics MQTT distincts**. Cette architecture Publish/Subscribe garantit un **couplage faible** entre les composants : Node-RED et l'agent LLM peuvent s'abonner indépendamment aux topics dont ils ont besoin, sans modifier le code de l'ESP32.

Ce chapitre décrit la mise en place de la couche de visualisation et de stockage historique de la serre agricole connectée. Après la configuration des capteurs, de la logique automatique et de la communication MQTT, cette partie exploite les données envoyées par l'ESP32 afin de les afficher en temps réel et de les sauvegarder dans une base de données locale.

Le rôle de cette tâche est double : d'une part, Node-RED reçoit les messages MQTT publiés par l'ESP32 simulé sous Wokwi ; d'autre part, les données sont affichées dans un tableau de bord interactif et insérées automatiquement dans une base SQLite afin de conserver un historique des mesures.

4.1 Mise en place de l'environnement Node-RED

Node-RED a été utilisé comme outil principal pour la réception des données MQTT, la construction du dashboard et l'interaction avec la base SQLite. Après installation sur la machine locale, l'interface est accessible depuis le navigateur à l'adresse :

`http://127.0.0.1:1880`

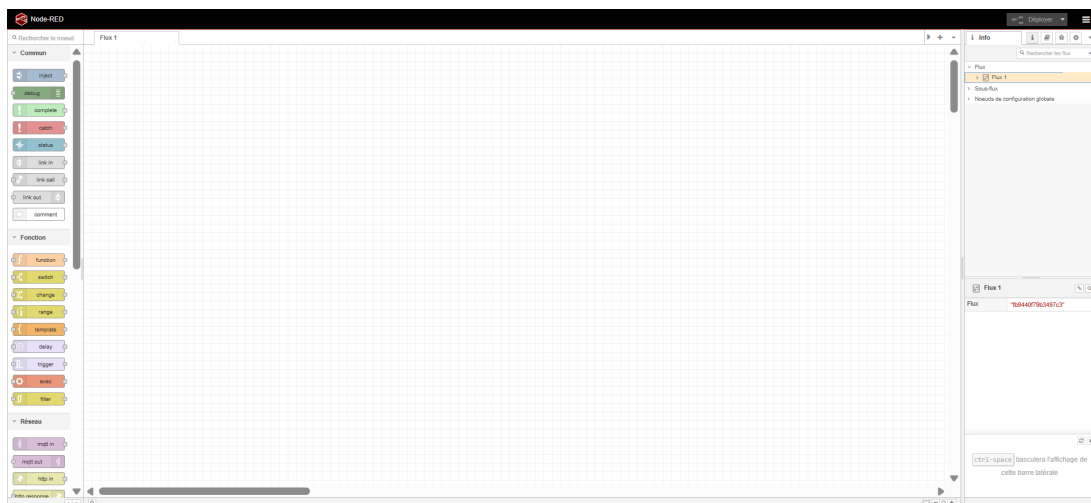


IMAGE 11 – Interface principale de Node-RED.

Les nœuds utilisés dans ce projet sont :

Nœud	Rôle
mqtt in	Réception des messages publiés par l'ESP32
json	Conversion des messages en objets JavaScript exploitables
function	Extraction et préparation des valeurs à afficher ou à stocker
debug	Vérification des messages reçus
gauge	Affichage des mesures sous forme de jauges
text	Affichage des états des actionneurs
chart	Affichage des courbes historiques
sqlite	Stockage des mesures dans une base de données locale

TABLE 9 – Nœuds Node-RED utilisés dans le projet.

4.2 Réception des données MQTT

La première étape a consisté à connecter Node-RED au broker MQTT utilisé par l'ESP32. Les paramètres de connexion sont récapitulés dans le tableau 10.

Paramètre	Valeur
Broker MQTT	broker.hivemq.com
Port	1883
Topic	serre/data/json
Format	JSON

TABLE 10 – Configuration MQTT utilisée dans Node-RED.

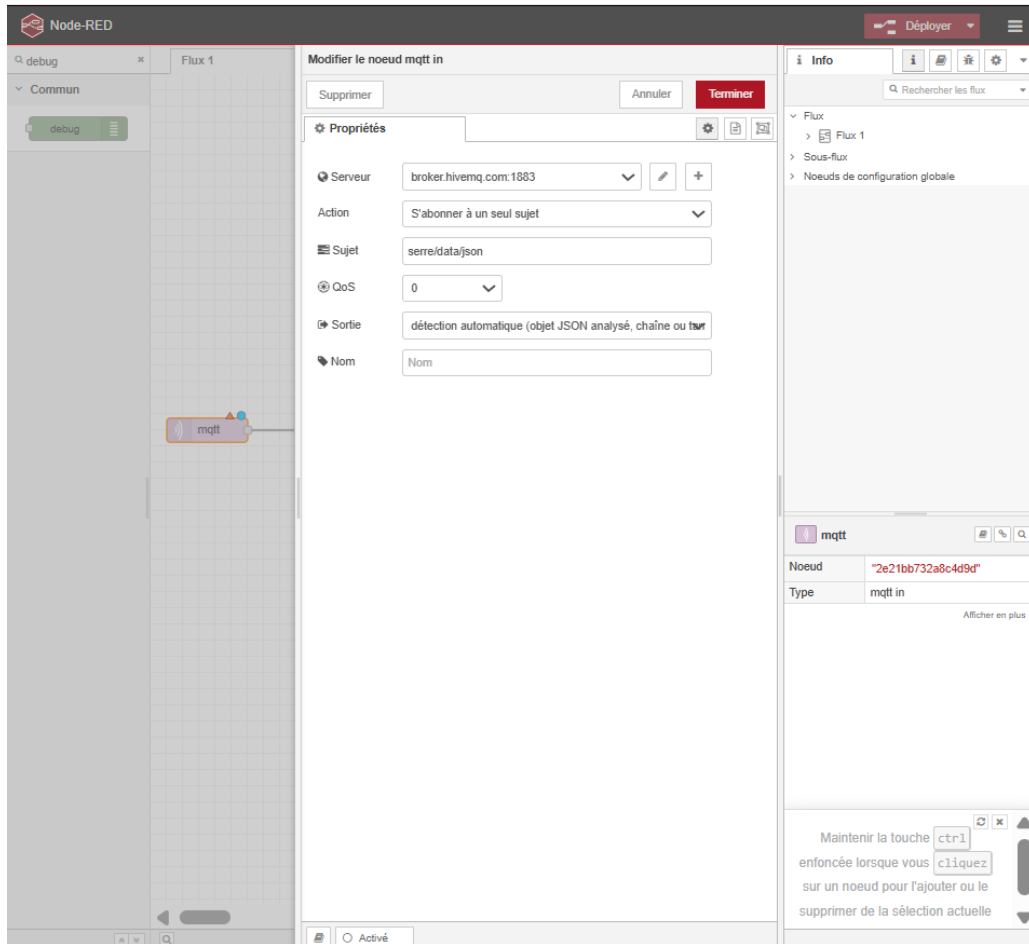


IMAGE 12 – Configuration du nœud MQTT In dans Node-RED.

Le flux de réception initial suit la chaîne : `mqtt in` → `json` → `debug`, permettant de vérifier que les données arrivent correctement.

4.3 Vérification et débogage des données reçues

Les données reçues ont été visualisées à l'aide du nœud `debug`. Un exemple de message JSON reçu depuis l'ESP32 est présenté ci-dessous :

```

1 {
2   "temp_air"    : 25.2,  "hum_air"    : 59.8,
3   "temp_sol"   : 10.1,  "hum_sol"   : 50.2,
4   "qualite_air": 540,   "niveau_eau": 10.1,
5   "ldr"        : 7,     "pompe"      : "OFF",
6   "led_verte"  : "OFF", "led_bleue" : "ON",
7   "led_rouge"  : "OFF", "meteo"      : "NUIT"
8 }

```

Listing 13 – Exemple de message JSON reçu depuis l'ESP32

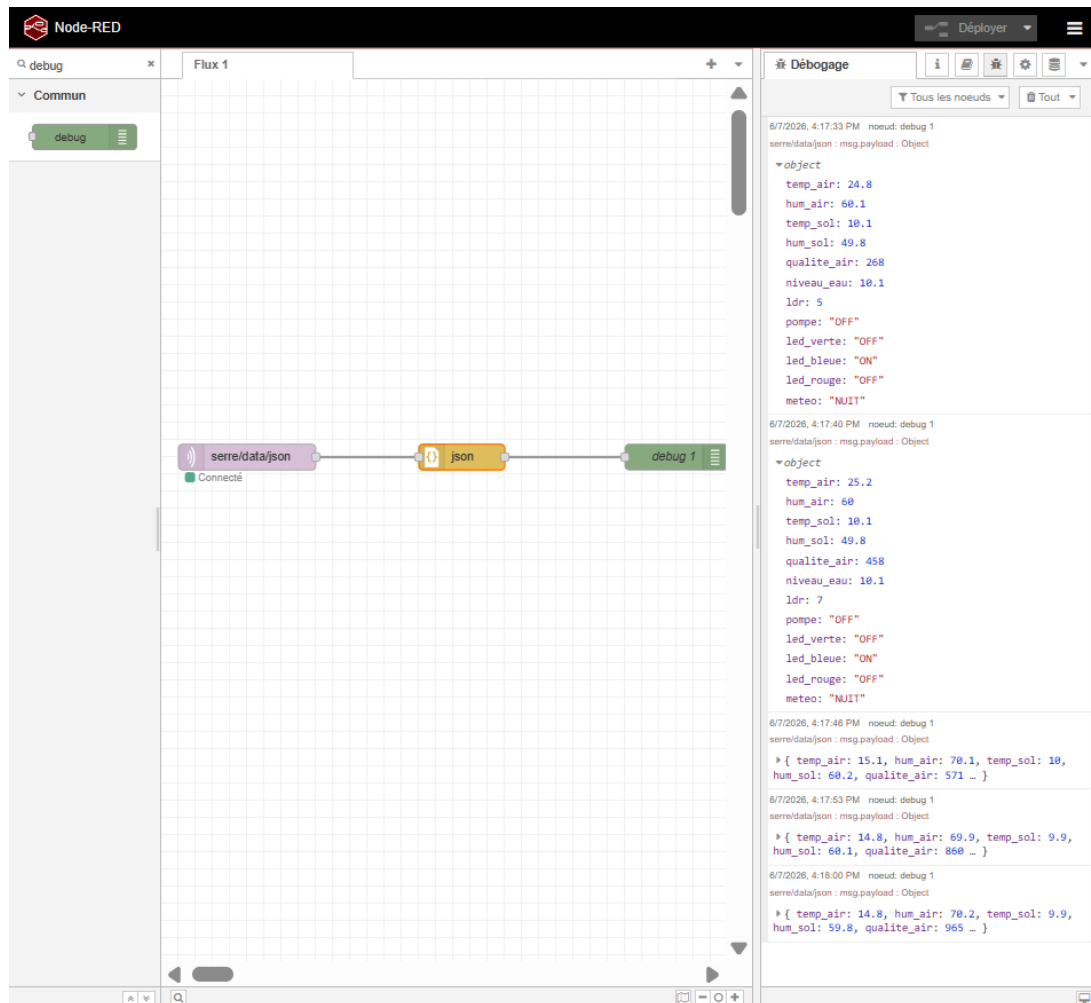


IMAGE 13 – Données reçues et affichées dans le panneau de débogage Node-RED.

4.4 Construction du dashboard Node-RED

Le dashboard est organisé en trois parties : les **conditions climatiques**, les **états des actionneurs** et les **graphiques historiques**.

4.4.1 Affichage des conditions climatiques

Les mesures numériques sont affichées sous forme de jauges. Chaque jauge reçoit une valeur extraite du message JSON via un nœud `function`; par exemple, pour la température :

```

1 msg.payload = msg.payload.temp_air;
2 return msg;

```

Listing 14 – Extraction de la température de l'air

Le même principe est appliqué à l'humidité de l'air, à la température et à l'humidité du sol, à la qualité de l'air, à la luminosité et au niveau d'eau.

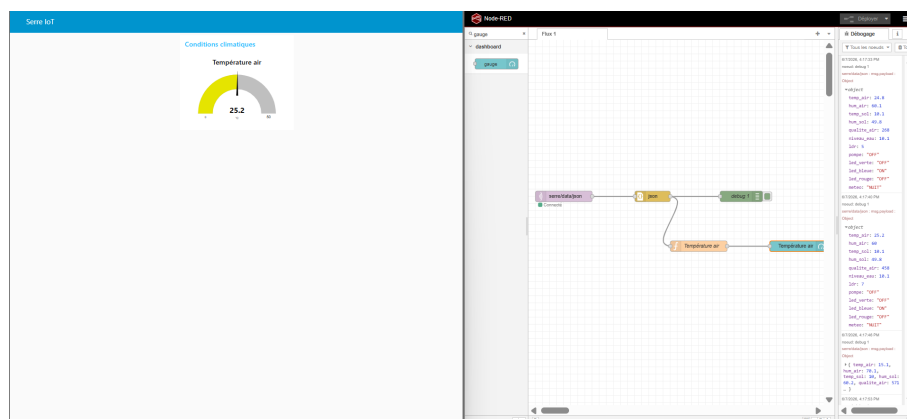


IMAGE 14 – Exemple de jauge affichant la température de l'air.

Une première version du dashboard a été réalisée pour valider l'affichage, puis améliorée afin d'obtenir une interface plus lisible.

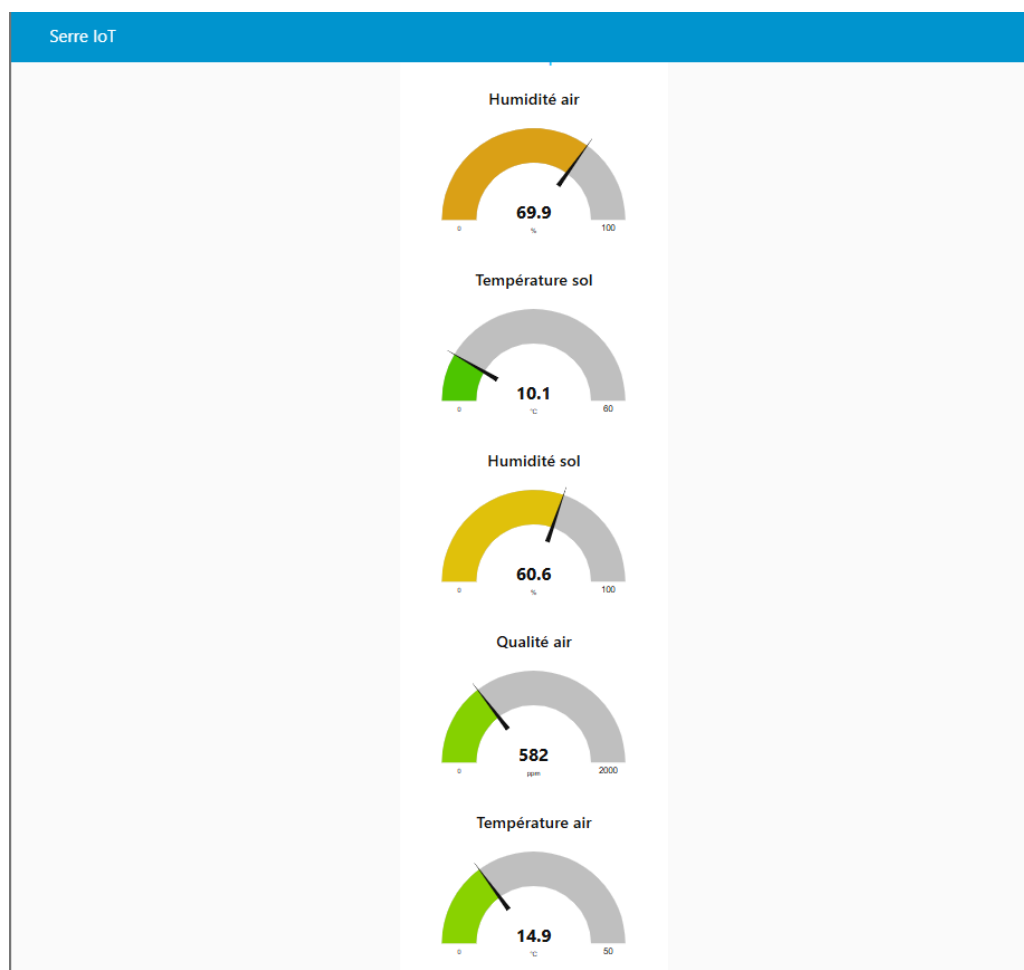


IMAGE 15 – Première version provisoire du dashboard Node-RED.

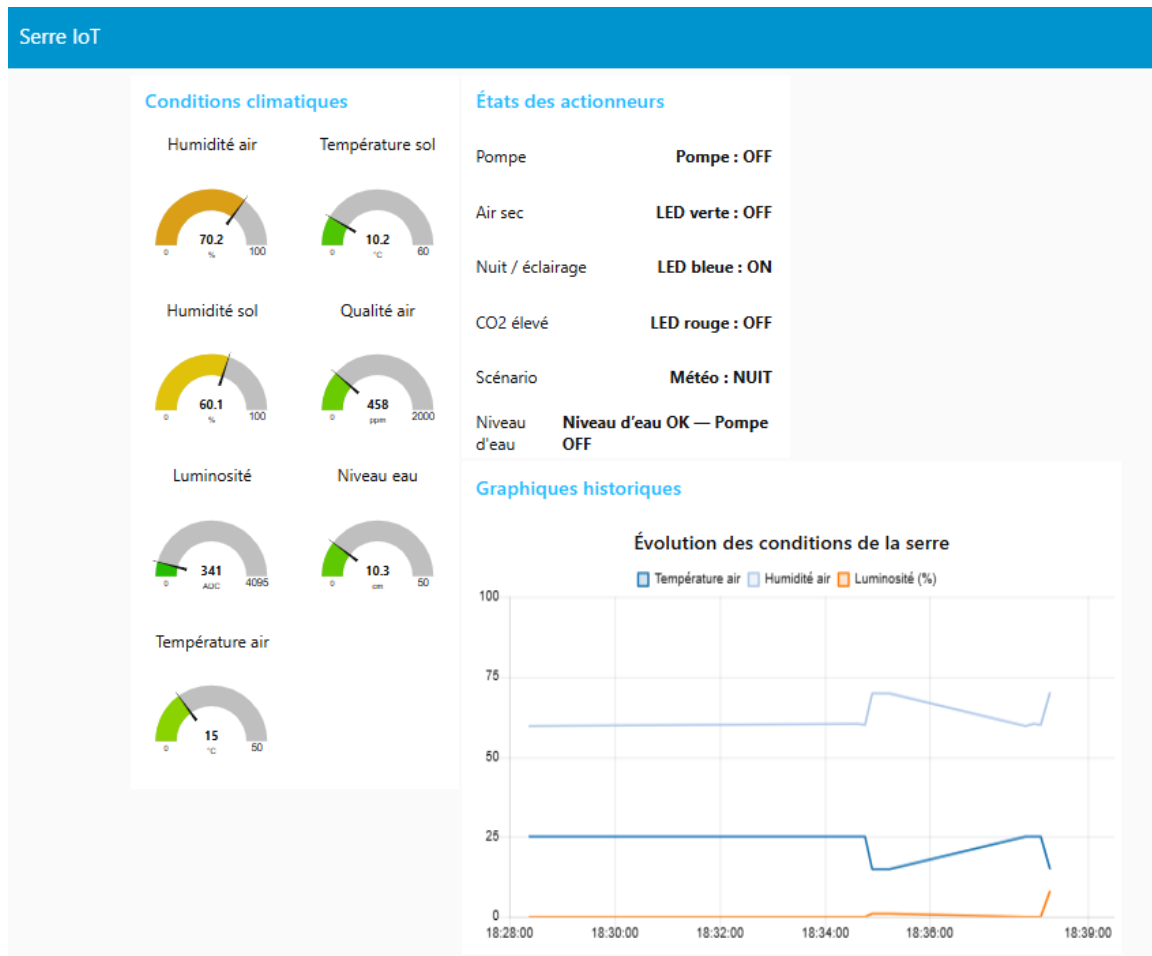


IMAGE 16 – Dashboard Node-RED final de la serre connectée.

4.4.2 Affichage des états des actionneurs

Le dashboard affiche également l'état des actionneurs : pompe, LED verte (air sec), LED bleue (nuit), LED rouge (CO₂), scénario météo et niveau d'eau. Pour ce dernier, une logique explicite produit un message compréhensible :

```

1 let distance = msg.payload.niveau_eau;
2 let pompe    = msg.payload.pompe;
3 if (distance > 20)
4     msg.payload = "Niveau d'eau faible -- Pompe " + pompe;
5 else
6     msg.payload = "Niveau d'eau OK -- Pompe " + pompe;
7 return msg;

```

Listing 15 – Interprétation du niveau d'eau

4.5 Graphique historique multi-courbes

Un graphique multi-courbes permet de visualiser simultanément la température de l'air, l'humidité de l'air et la luminosité normalisée. Trois nœuds `function` alimentent le même nœud `chart`, chaque courbe étant identifiée par le champ `msg.topic` :

```

1 // Courbe temperature
2 msg.payload = msg.payload.temp_air;
3 msg.topic   = "Temperature air";
4 return msg;
5
6 // Courbe humidite
7 msg.payload = msg.payload.hum_air;
8 msg.topic   = "Humidite air";
9 return msg;
10
11 // Luminosite normalisee (ADC 0-4095 -> 0-100 %)
12 msg.payload = Math.round((msg.payload.ldr / 4095) * 100);
13 msg.topic   = "Luminosite (%)";
14 return msg;

```

Listing 16 – Préparation des trois courbes du graphique

4.6 Stockage historique avec SQLite

Chaque mesure reçue est automatiquement insérée dans une base SQLite nommée `serre.db`, table `mesures`.

4.6.1 Création de la table

Un flux dédié (`inject` → `function` → `sqlite`) initialise la table à la première exécution :

```

1 CREATE TABLE IF NOT EXISTS mesures (
2     id            INTEGER PRIMARY KEY AUTOINCREMENT,
3     timestamp     DATETIME DEFAULT CURRENT_TIMESTAMP,
4     temp_air      REAL,   hum_air      REAL,
5     temp_sol      REAL,   hum_sol      REAL,
6     qualite_air   REAL,   niveau_eau  REAL,
7     ldr           INTEGER,
8     pompe         TEXT,   led_verte  TEXT,
9     led_bleue     TEXT,   led_rouge TEXT,
10    meteo          TEXT
11 );

```

Listing 17 – Création de la table mesures

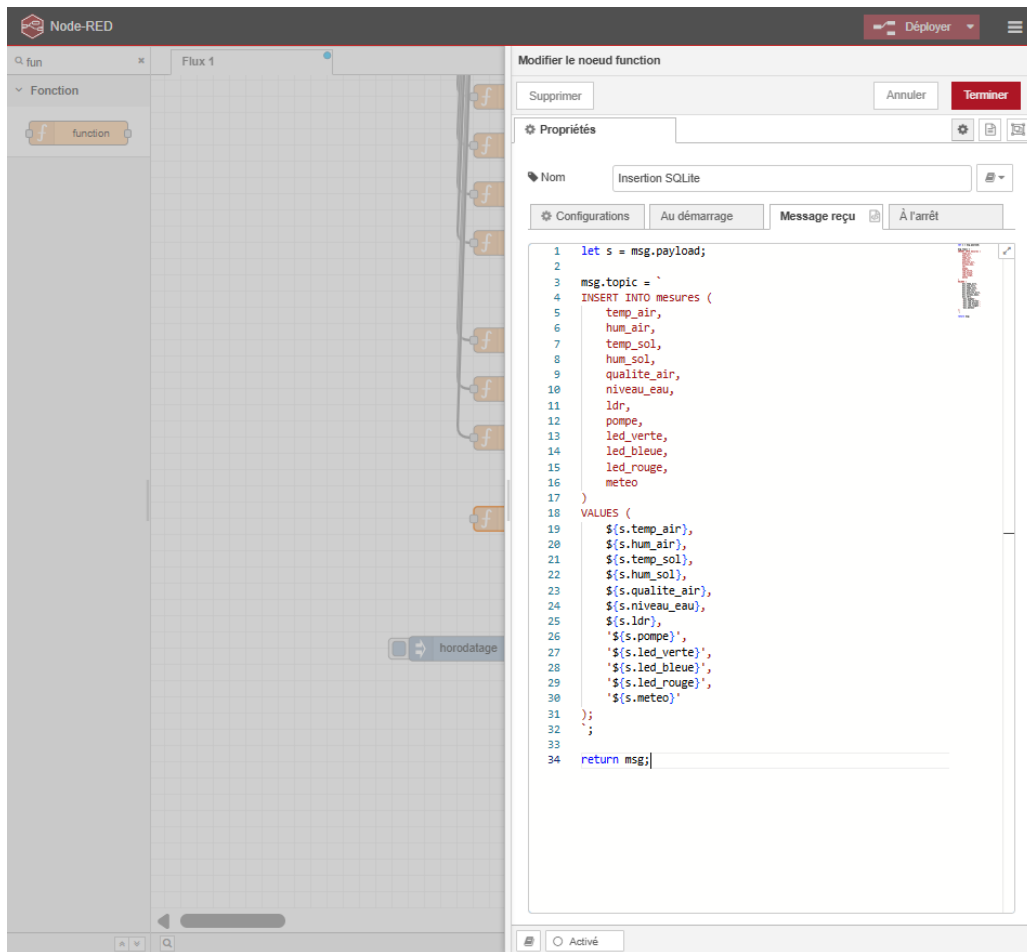


IMAGE 17 – Configuration de l’insertion SQLite dans Node-RED.

4.6.2 Insertion automatique des mesures

À chaque réception d’un message MQTT, Node-RED construit et exécute la requête d’insertion :

```

1 let s = msg.payload;
2 msg.topic = 'INSERT INTO mesures (
3     temp_air, hum_air, temp_sol, hum_sol,
4     qualite_air, niveau_eau, ldr,
5     pompe, led_verte, led_bleue, led_rouge, meteo)
6 VALUES (
7     ${s.temp_air}, ${s.hum_air}, ${s.temp_sol}, ${s.hum_sol},
8     ${s.qualite_air}, ${s.niveau_eau}, ${s.ldr},
9     '${s.pompe}', '${s.led_verte}', '${s.led_bleue}',
10    '${s.led_rouge}', '${s.meteo}')';
11 return msg;

```

Listing 18 – Insertion automatique des mesures dans SQLite

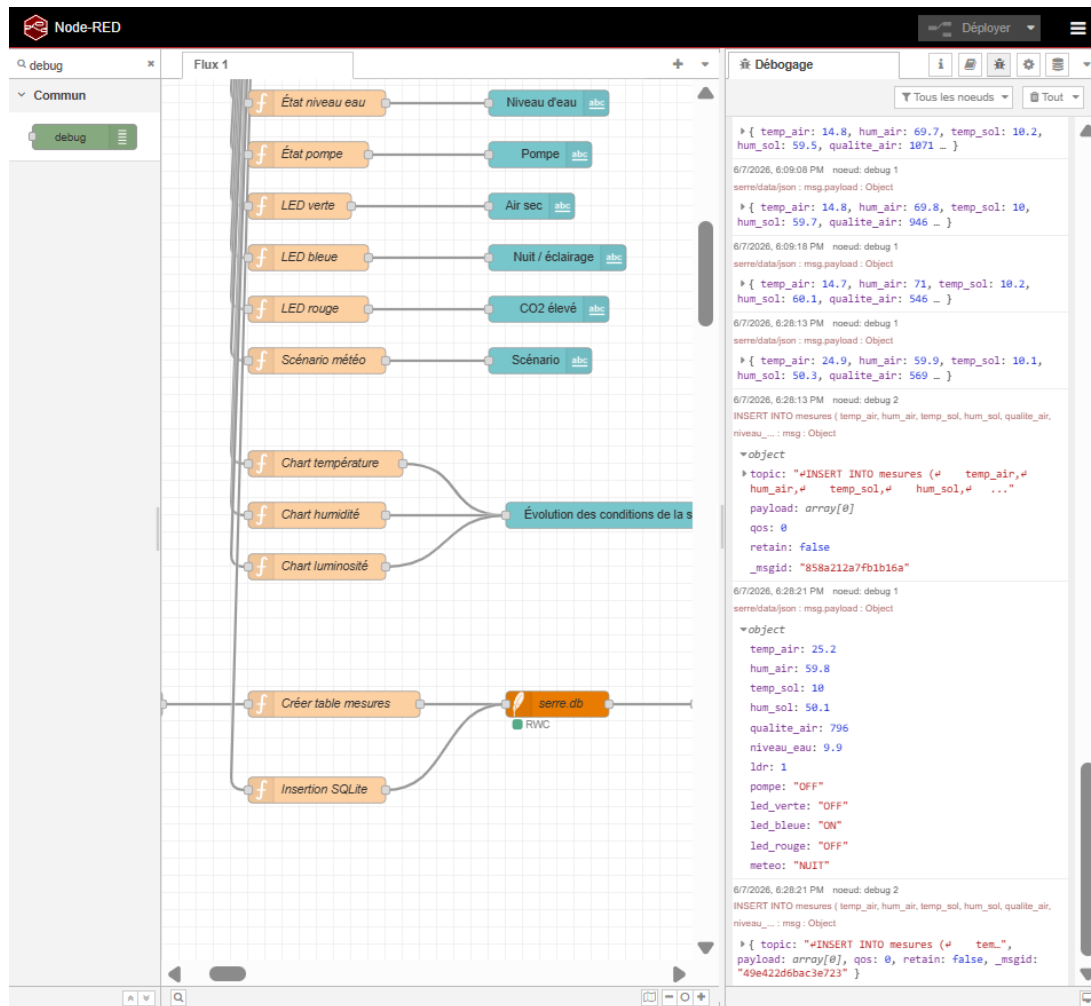


IMAGE 18 – Insertion automatique des mesures dans la base SQLite.

4.6.3 Vérification des données stockées

Une requête `SELECT` permet de contrôler les enregistrements :

```
1 SELECT * FROM mesures ORDER BY id DESC LIMIT 10;
```

Listing 19 – Consultation des dernières mesures

4.7 Organisation générale du flux Node-RED

Le flux final est organisé en plusieurs branches partant du nœud `json` : débogage, jauges, états textuels, graphique multi-courbes, insertion SQLite et consultation des mesures stockées.

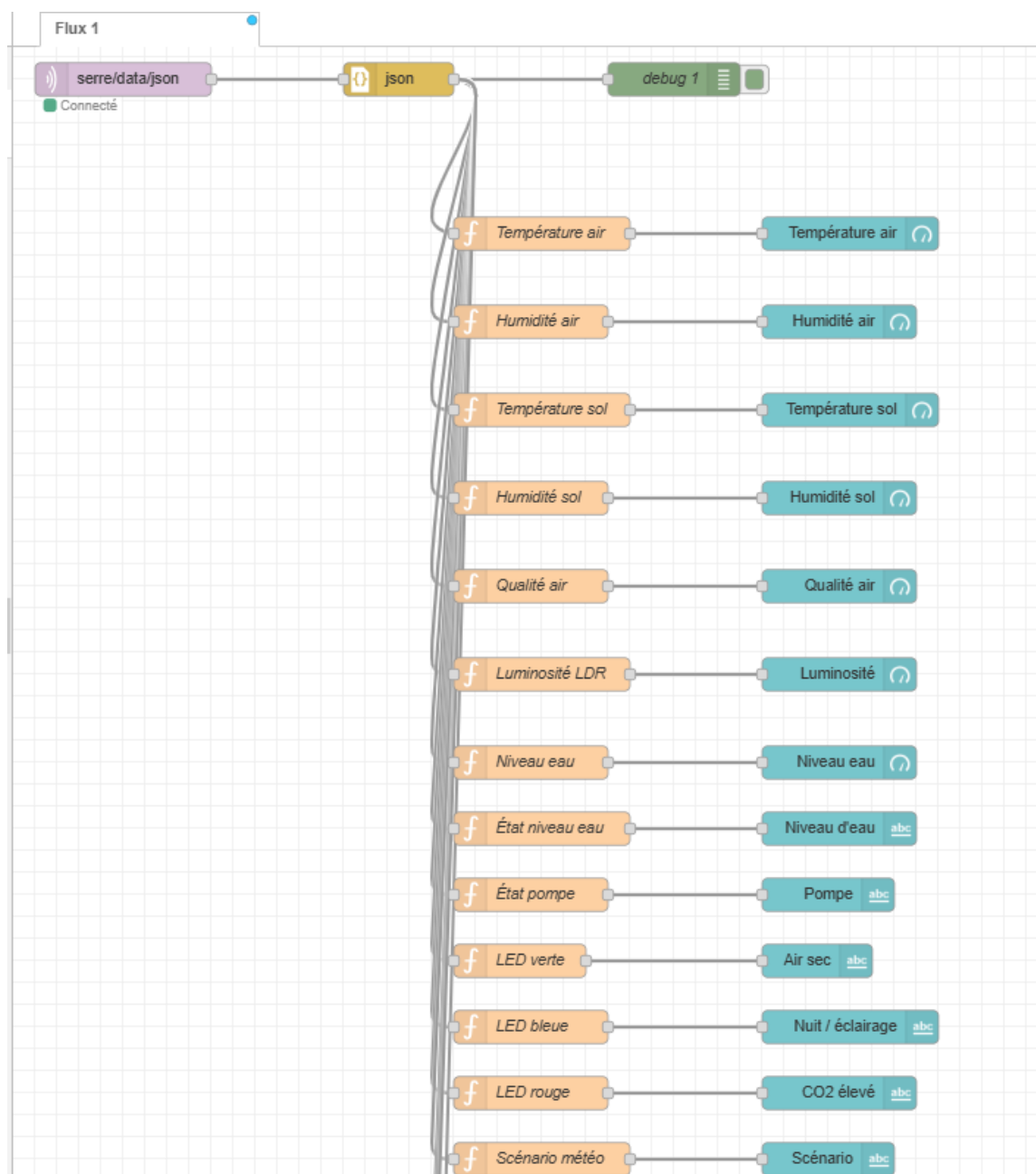


IMAGE 19 – Vue partielle du flux Node-RED : jauges et états des actionneurs.

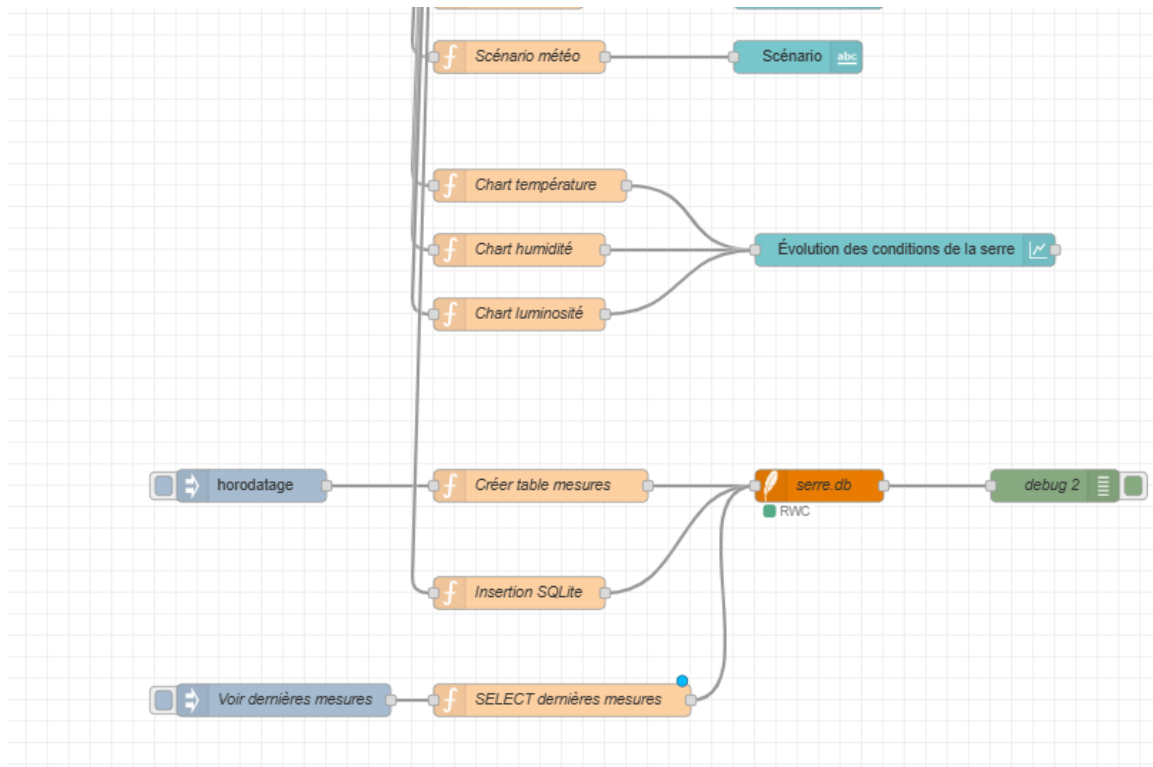


IMAGE 20 – Vue partielle du flux Node-RED : graphiques et stockage SQLite.

Conclusion du chapitre

Ce chapitre a mis en place la couche de **supervision et de mémorisation** du projet : connexion au broker HiveMQ, affichage en temps réel via des **jauges** et un **graphique multi-courbes**, et stockage structuré dans une base **SQLite**. L'historique ainsi constitué fournit au chapitre 5 les données nécessaires à l'analyse intelligente par l'**agent LLM**.

5.1 Vue d'ensemble

Cette tâche regroupe deux responsabilités complémentaires : la mise en place du **serveur backend Flask** assurant la réception des données MQTT, leur persistance et l'exposition des routes REST ; et l'intégration de l'**Agent d'Intelligence Artificielle SmartAssist**, un conseiller agronomique conversationnel basé sur un grand modèle de langage (LLM).

5.2 Partie A — Backend Flask

5.2.1 Rôle du backend

Le serveur Flask constitue le **nœud central** du pipeline IoT. Il assure quatre fonctions essentielles :

1. **Réception MQTT** : souscription au topic `serre/tomates/data` via `paho-mqtt`.
2. **Persistance** : insertion automatique de chaque relevé dans la base SQLite (`serre.db`), table `mesures`.
3. **Exposition REST** : routes HTTP consommées par le dashboard React et Node-RED.
4. **Calcul du score de santé** : algorithme heuristique évaluant l'état global de la serre à chaque nouveau relevé.

5.2.2 Routes REST exposées

Route	Méthode	Description
<code>/status</code>	GET	Retourne le dernier relevé + score de santé + alertes actives
<code>/history</code>	GET	Retourne les N dernières mesures stockées en SQLite
<code>/update</code>	POST	Réception des données capteurs depuis Node-RED
<code>/analyze</code>	GET	Interrogation de l'agent IA avec une question agronomique
<code>/cmd</code>	POST	Envoi d'une commande actionneur (éclairage, arrosage)

5.2.3 Algorithme de calcul du score de santé

Paramètre	Seuil min	Seuil max	Idéal
Température	18 °C	28 °C	22 °C
Humidité	60%	80%	70%
Luminosité	400 lux	—	> 600 lux

Le score final génère un niveau qualitatif : *Très bon* (≥ 80), *Moyen* (60–79), ou *Critique* (< 60).

5.3 Partie B — Architecture Globale — Pipeline à Quatre Couches

Couche	Description
Terrain (IoT)	Capteurs DHT22 + LDR publient sur le topic MQTT serre/tomates/data
Backend (Flask)	Réception, stockage SQLite, calcul du Health Score, routes REST
Cognitive (IA)	Contexte enrichi RAG transmis au LLM Groq Llama-3.3-70B
Présentation (React)	Dashboard React + chat conversationnel SmartAssist

5.4 Partie C — Intégration de l’Agent LLM SmartAssist

5.4.1 Pipeline d’intégration IA — Étape par Étape

Étape 1 : Acquisition et notification (IoT & MQTT) Les capteurs publient leurs relevés en continu sur le topic `serre/tomates/data`.

Étape 2 : Persistance et calcul de l’indice de santé Le backend effectue l’insertion automatique dans SQLite et le calcul du score de santé heuristique.

Étape 3 : Génération de contexte enrichi — Approche RAG

1. **Extraction du contexte** : interrogation de SQLite pour extraire la moyenne des 12 dernières heures.

2. **Construction du prompt système** : assemblage d'une invite combinant le rôle de l'IA, les données physiques réelles, les pénalités en cours, le score de santé et l'historique récent de la discussion.
3. **Inférence LLM** : ce bloc de contexte enrichi est transmis à l'API Groq.

Étape 4 : Rendu visuel et actionneur (Frontend React) Le tableau de bord React interroge le backend via `GET /status` pour mettre à jour les jauges et l'anneau SVG du score de santé.

5.5 Interface du Dashboard — Vue en Temps Réel

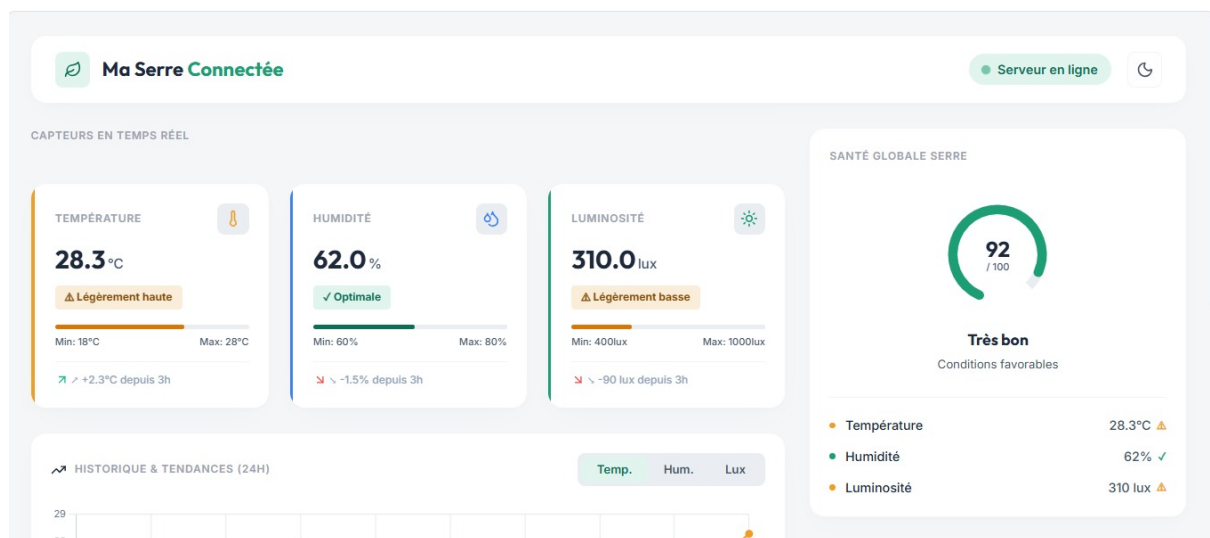


IMAGE 21 – Dashboard React — Vue d'ensemble : capteurs en temps réel et score de santé global (Score : 92/100 — Très bon)

5.6 Interface SmartAssist — Agent Conversationnel

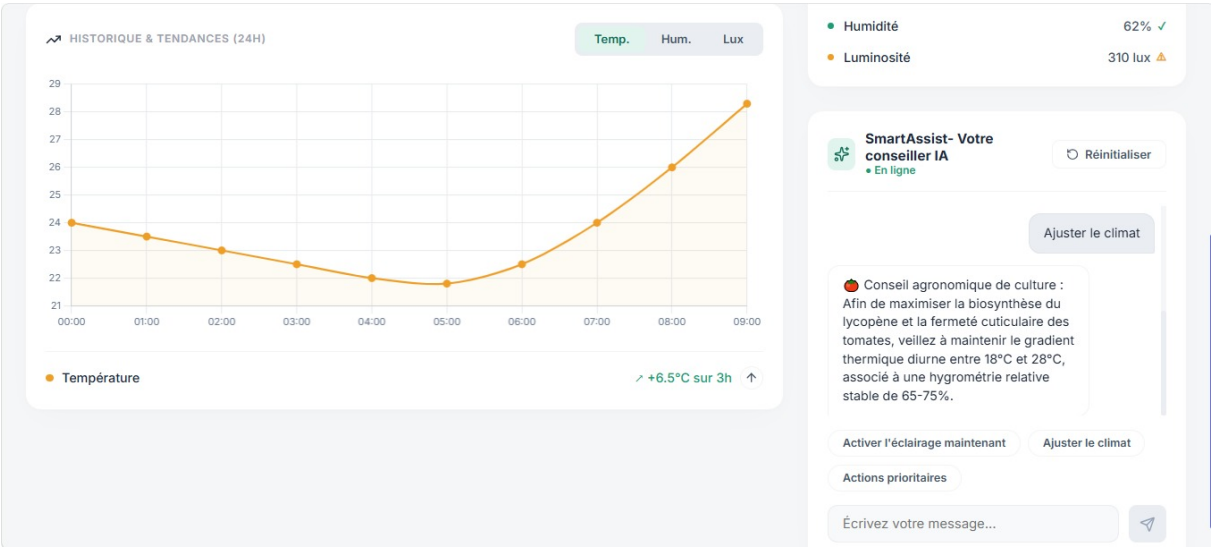


IMAGE 22 – Dashboard React — Panneau SmartAssist avec conseil agronomique en temps réel

5.7 Scénario d’exécution réel — Diagnostic de luminosité

Paramètre	Valeur	Seuil	Statut	Action IA
Température	28,3 °C	18–28 °C	⚠ Haute	Surveillance climat
Humidité	62,0%	60–80%	✓ OK	Aucune
Luminosité	310 lux	> 400 lux	✗ Insuffisant	Activation LEDs

5.8 Avantages et Plus-value

1. **Prévention des risques** : évaluation en temps réel, avant tout signe physique.
2. **Vulgarisation de la donnée** : l’exploitant échange en langage naturel.
3. **Automatisation intelligente** : pont direct entre analyse et commandes d’actionneurs.
4. **Approche RAG sans hallucination** : le LLM s’appuie exclusivement sur les données réelles issues de SQLite.

Conclusion générale

Ce projet a abouti à la réalisation d'une serre agricole connectée et automatisée, combinant acquisition de données en temps réel (ESP32 et une chaîne de six capteurs), logique de contrôle automatique (pompe d'arrosage et indicateurs lumineux), communication MQTT, visualisation Node-RED, persistance SQLite et analyse intelligente par agent LLM.

Les choix technologiques — Wokwi pour la simulation, MQTT pour la communication, Node-RED pour le dashboard, SQLite pour l'historique et l'intégration LLM — répondent aux objectifs de surveillance en temps réel, d'automatisation et de conseil intelligent fixés au départ.

Trois pistes concrètes d'évolution restent ouvertes : déployer le système sur un dispositif physique réel, étendre l'agent LLM à d'autres cultures, et ajouter une interface mobile permettant de recevoir des alertes depuis n'importe où.

Références et supports utilisés

Plateformes et environnements

- **Wokwi** — Simulateur d'électronique et de microcontrôleurs (ESP32) : <https://wokwi.com>
- **Tinkercad Circuits** — Plateforme de simulation comparée : <https://www.tinkercad.com>
- **Arduino IDE / Arduino-ESP32** — Framework de développement pour ESP32 : <https://www.arduino.cc>
- **Node-RED** — Outil de dashboard et de flux de données : <https://nodered.org>

Bibliothèques logicielles

- **DHT sensor library** (Adafruit) — lecture du capteur DHT22.
- **Adafruit GFX Library** et **Adafruit SSD1306** — pilotage de l'écran OLED.
- **OneWire** et **DallasTemperature** — sondes de température (DS18B20).
- **PubSubClient** — client MQTT pour l'ESP32.
- **WiFi** (ESP32) — connectivité réseau.

Documentation technique des composants

- Microcontrôleur **ESP32** (Espressif Systems).
- Capteur de température et d'humidité **DHT22 / AM2302**.
- Capteur ultrason **HC-SR04**.
- Capteur de gaz **MQ135**.
- Afficheur **OLED SSD1306** (I²C, 128×64).
- Thermistance **NTC** et photorésistance **LDR**.